```
clc
clear

%Calculating the Transformaion matrix for each joint

use_syms = false;


if  use_syms
   % to get symbollic answer
   syms q1 q2 q3 q4 q5;
else
    % to get a numeric answer
    q = [0 0 0 0 0 0];
    q1 = q(1);
    q2 = q(2);
    q3 = q(3);
    q4 = q(4);
    q5 = q(5);
end

% Lynx ADL5 constants in mm
d1 = 76.2;                      % Distance between joint 0 and joint 1
a2 = 146.05;                    % Distance between joint 1 and joint 2
a3 = 187.325;                   % Distance between joint 2 and joint 3
d4 = 34;                        % Distance between joint 3 and joint 4
d5 = 68;                        % Distance between joint 3 and joint
 end-effector



%Calculating the Tansformation matrix for the different frames of the
 robot
%Frame 1 w.r.t Frame 0
T01 = [cos(q1) -sin(q1)*cos(-pi/2)  sin(q1)*sin(-pi/2)  0;
      sin(q1)  cos(q1)*cos(-pi/2) -cos(q1)*sin(-pi/2)  0;
               0            sin(-pi/2)            cos(-pi/2) d1;
               0                    0                  0     1];

%Cleaned Matrix for symbollic calculation
T01_c = [cos(q1) 0  -sin(q1)  0;
      sin(q1) 0 cos(q1)  0;
               0            -1            0 d1;
               0                    0                  0     1];


%Frame 2 w.r.t Frame 1
T12 = [cos(q2-(pi/2)) -sin(q2-(pi/2))  0   a2*cos(q2-(pi/2));
      sin(q2-(pi/2))  cos(q2-(pi/2))  0   a2*sin(q2-(pi/2));
               0                       0  1                    0;
               0                       0  0                    1];
```

```
%Cleaned Matrix for symbollic calculation
T12_c = [sin(q2) cos(q2)  0   a2*sin(q2);
      -cos(q2)  sin(q2)  0   -a2*cos(q2);
              0                       0  1                     0;
              0                       0  0                     1];

%Frame 3 w.r.t Frame 2
T23 = [cos(q3+(pi/2)) -sin(q3+(pi/2))  0   a3*cos(q3+(pi/2));
      sin(q3+(pi/2))  cos(q3+(pi/2))  0   a3*sin(q3+(pi/2));
              0                       0  1                     0;
              0                       0  0                     1];

%Cleaned Matrix for symbollic calculation
T23_c = [-sin(q3) -cos(q3)  0   -a3*sin(q3);
      cos(q3)  -sin(q3)  0   a3*cos(q3);
              0                       0  1                     0;
              0                       0  0                     1];

%Frame 4 w.r.t Frame 3
T34 = [cos(q4-(pi/2)) -sin(q4-(pi/2))*cos(-pi/2)   sin(q4-
(pi/2))*sin(-pi/2)   0;
      sin(q4-(pi/2))  cos(q4-(pi/2))*cos(-pi/2)  -cos(q4-(pi/2))*sin(-
pi/2)   0;
              0                          sin(-pi/2)
 cos(-pi/2)   0;
              0                                  0
          0   1];

%Cleaned Matrix for symbollic calculation
T34_c = [sin(q4) 0   cos(q4)   0;
      -cos(q4)  0  sin(q4)   0;
              0                          -1                    0   0;
              0                                  0
          0   1];
%End-effector w.r.t Frame 4
T4e = [cos(q5) -sin(q5)  0        0;
      sin(q5)  cos(q5)  0        0;
              0          0  1       d5;
              0          0  0        1];

%Cleaned Matrix for symbollic calculation
T4e_c = [cos(q5) -sin(q5)  0        0;
      sin(q5)  cos(q5)  0        0;
              0          0  1       d5;
              0          0  0        1];

% Computing the Transformation matrix to Frame 0
T02 = T01_c*T12_c;
T03 = T02*T23_c;
T04 = T03*T34_c;
T0e = T04*T4e_c;

%Calculating the Zaxis of frame i wrt to Frame 0
z_0_i = [[0 0 1];
```

```
            T01_c([9 10 11]);
            T02([9 10 11]);
            T03([9 10 11]);
            T04([9 10 11])
            T0e([9 10 11])];

%Calculating the position of orgin of each joint

%Position of Third Joint (Elbow Revolute)
X(3,:) = (T02*[0;0;0;1])';

%Position of First Joint (Base Revolute)
X(1,:) = [0 0 0 1];

%Position of Second Joint (Shoulder Revolute)
X(2,:) = (T01_c*[0;0;0;1])';

%Position of Fourth Joint (1st Wrist)
X(4,:) = (T03*[0;0;0;1])';

%Position of Fifth Joint (2nd Wrist)
X(5,:) = (T04*[0;0;d4;1])';

%Position of Gripper (Base of the Gripper)
X(6,:) = (T0e*[0;0;0;1])';


%Calculating the locations of each joint in the Base Frame
jointPositions = X(:,1:3);

%Calculating linear joint velocity
%Using formuala for Revolute joints : Jv = Z_i-1 x (O_6 - O_i-1)
J_v(:,1) = cross(z_0_i(1,:),(jointPositions(6,:)-
jointPositions(1,:)))';
J_v(:,2) = cross(z_0_i(2,:),(jointPositions(6,:)-
jointPositions(2,:)))';
J_v(:,3) = cross(z_0_i(3,:),(jointPositions(6,:)-
jointPositions(3,:)))';
J_v(:,4) = cross(z_0_i(4,:),(jointPositions(6,:)-
jointPositions(4,:)))';
J_v(:,5) = cross(z_0_i(5,:),(jointPositions(6,:)-
jointPositions(5,:)))';
J_v(:,6) = cross(z_0_i(6,:),(jointPositions(6,:)-
jointPositions(6,:)))';

%Input of Joint velocities
q_dot(:,1) = [0 0 0 0 0 0];
q_dot(:,2) = [0.1 0 0 0 0 0];
q_dot(:,3) = [0 0.1 0 0 0 0];
q_dot(:,4) = [0 0 0.1 0 0 0];
q_dot(:,5) = [0 0 0 0.1 0 0];
q_dot(:,6) = [0 0 0 0 0.1 0];
q_dot(:,7) = [0 0 0 0 0 0.1];
```

```
%Compiling Linear velocity
v = (J_v*q_dot())'
```

```
v =

         0         0         0
         0   25.5325         0
   14.6050         0  -25.5325
         0         0  -25.5325
         0         0   -6.8000
         0         0         0
         0         0         0
```

Published with MATLAB® R2020a